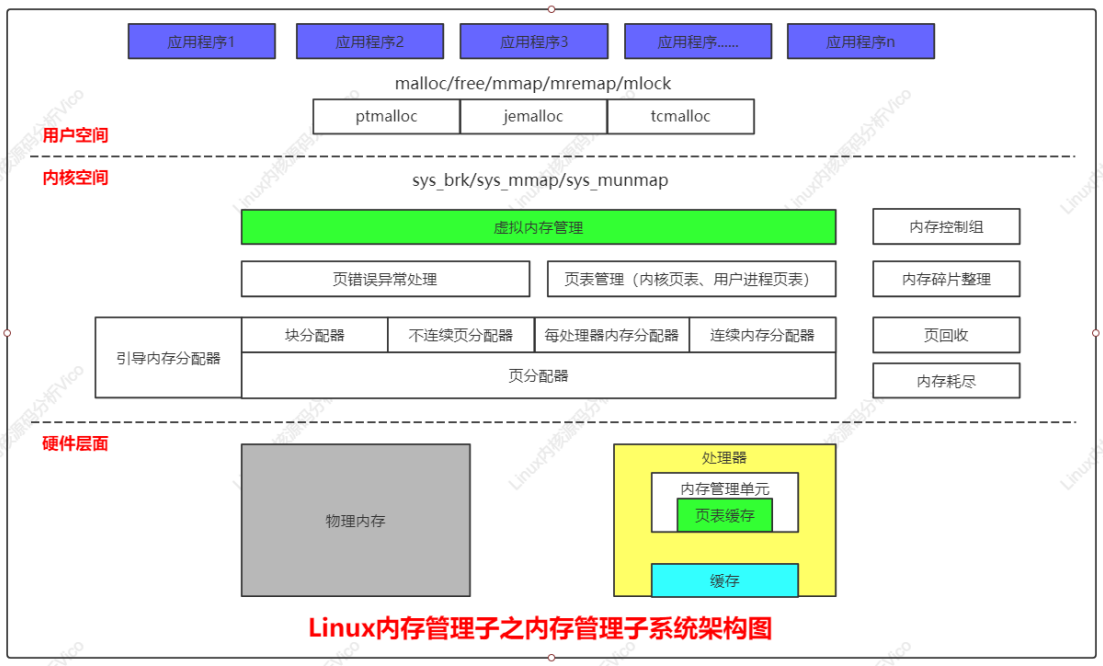
Linux 内核源码分析：内存管理子系统架构（用户空间、内核空间、硬件层面）



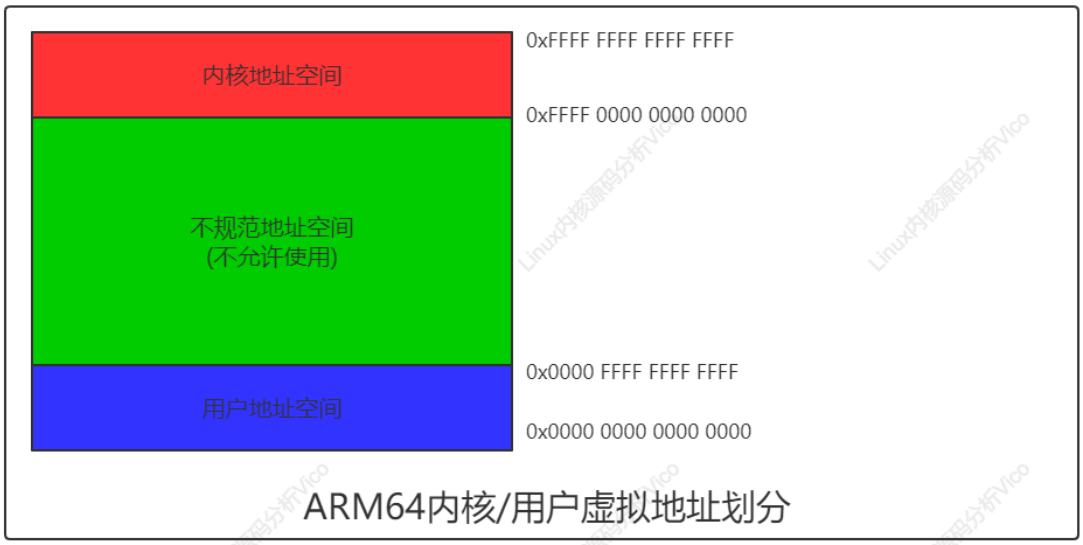
Linux内存管理子之内存管理子系统架构图

一、虚拟地址布局/内存映射

1、虚拟地址空间划分

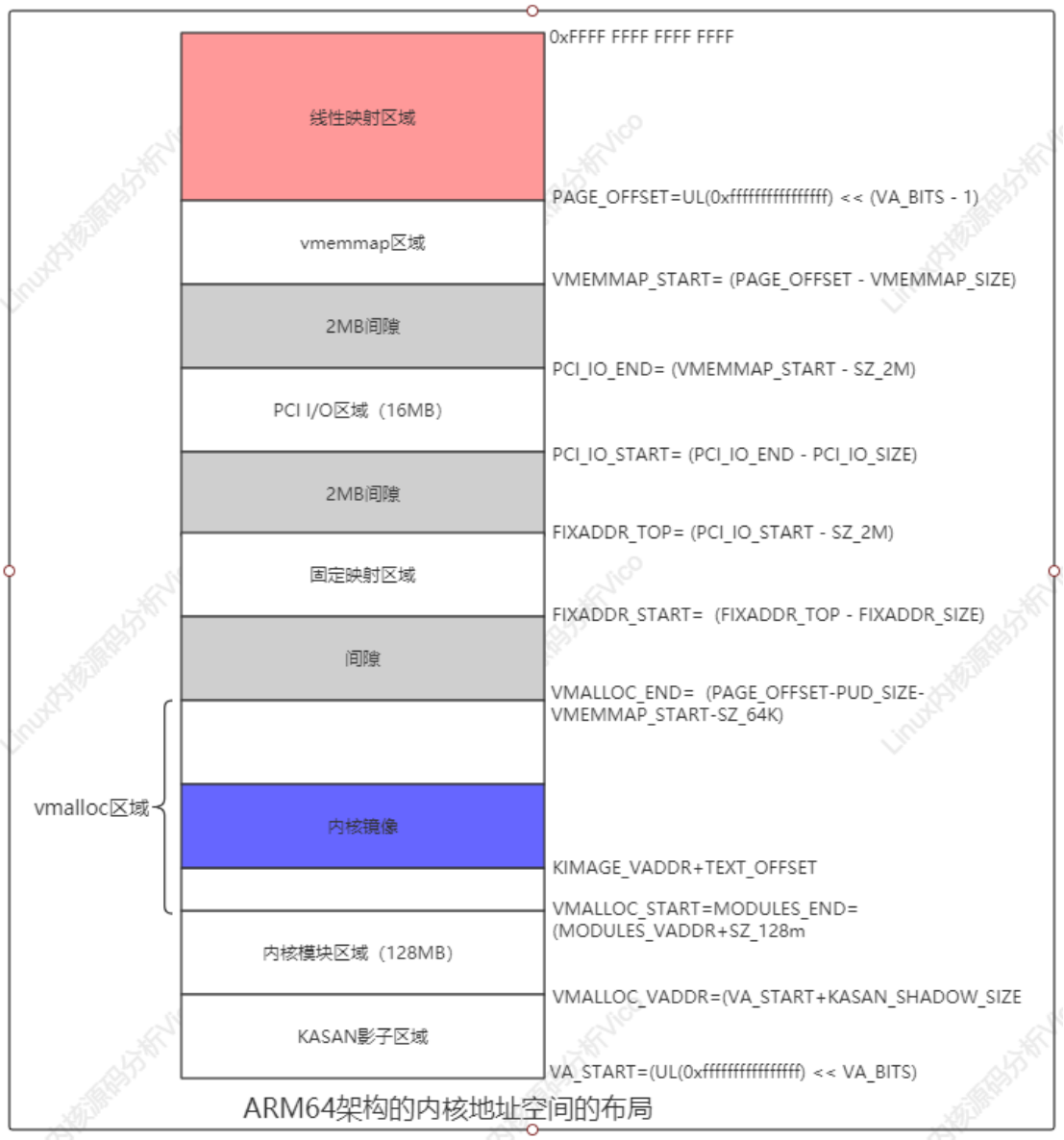
1）用户虚拟地址空间布局

目前应用程序没有那么大的内存需求、所以 ARM64 处理器不支持完全的 64 位虚拟地址，虚拟地址的最大宽度是 48 位，如下图所示：



2）内核地址空间布局

ARM64 处理器架构的内核地址空间布局如下图所示：



ARM64架构的内核地址空间的布局

2、物理地址空间

物理地址是处理器在系统总线上看到的地址。使用精简指令集（Reduced Instructions Set Computer，RISC）的处理顺路通常只一个物理地址空间，外围设备和物理内存使用统一的物理地址空间。有些处理顺路架构把分配给外围设备的物理地址区域称为设备内存。处

理器通过外围设备控制器的寄存器访问外围设备，寄存器分为控制寄存器、状态寄存器和数据寄存器三大类，外围设备的寄存器通常被连续编址。处理器对外围设备寄存器的编址方式有两种：I/O 映射方式（I/O-mapped）和内存映射方式（memory-mapped）。

程序只能通过虚拟地址访问外设寄存器，内核提供如下函数来把外设寄存器的物理地址映射到虚拟地址空间。

a. 函数 ioremap()把外设寄存器的物理地址映射到内核虚拟地址空间。

void * ioremap(unsigned long phys_addr, unsigned long size, unsigned long flags)

入口： phys_addr：要映射的起始的 IO 地址；

size：要映射的空间的大小；

flags：要映射的 IO 空间的和权限有关的标志；

phys_addr：是要映射的物理地址；

size：是要映射的长度，单位是字节

b. 函数 io_remap_pfn_range()把外设寄存器的物理地址映射到进程的用户虚拟地址空间。

int remap_pfn_range(struct vm_area_struct *vma, unsigned long virt_addr, unsigned long pfn, unsigned long size, pgprot_t prot);

int io_remap_pfn_range(struct vm_area_struct *vma, unsigned long virt_addr, unsigned long pfn, unsigned long size, pgprot_t prot);

remap_pfn_range 和 io_remap_pfn_range 负责为一段物理地址建立新的页表，他们的原型如下函数的返回值是 0 或者是个负的错误码。

第一个函数是在 pfn 指向实际的 RAM 的时候使用，后面一个函数实在 phy_addr 指向 I/O 内存时候使用，在实际使用中，除了 SPARC 外，对每个体系架构这两个函数都是等价的。大多数情况下使用第一个函数，对于有移植性的要求的驱动程序，要使用与特定情形相符的 remap_pfn_range 函数变种。

该函数的功能是创建页表。其中参数 vma 是内核根据用户的请求自己填写的，而参数 addr 表示内存映射开始处的虚拟地址，因此，该函数为 addr~addr+size 之间的虚拟地址构造页表。 另外，pfn（Page Fram Number）是虚拟地址应该映射到的物理地址的页

面号，实际上就是物理地址右移 PAGE_SHIFT 位。如果 PAGE_SHIFT 为 4kb，则 PAGE_SHIFT 为 12，因为 PAGE_SHIFT 等于 1<<PAGE_SHIFT。最一个参数 prot 是新页所要求的保护属性。　　在驱动程序中，一般能使用 remap_pfn_range（）映射内存中的保留页（如 X86 系统中的 640KB~1MB 区域）和设备 I/O 内存。因此，如果想把 kmalloc()申请的内存映射到用户空间，则可以通过 mem_map_reserve()把相应的内存设置为保留后就可以。

c. 内核提供函数 iounmap()，它用来删除函数 ioremap()创建的映射。

void * __ioremap(unsigned long phys_addr, unsigned long size, unsigned long flags)

void *ioremap(unsigned long phys_addr, unsigned long size)

入口： phys_addr：要映射的起始的 IO 地址；

size：要映射的空间的大小；

flags：要映射的 IO 空间的和权限有关的标志；

phys_addr：是要映射的物理地址

size：是要映射的长度，单位是字节

3、内存映射应用编程接口实战

内存映射是在进程的虚拟地址空间中创建一个映射，可分为两种：文件映射和匿名映射。通常把文件映射的物理页称为文件页，把匿名映射的物理页称为匿名页。

根据修改是否对其他进程 可见和是否传递到底层文件，内存映射分为共享映射和私有映射，两个进程可以使用共享的文件映射实现共享内存。匿名映射通常是私有映射，共享的匿名映射只可能出现在父进程和子进程之间。

在进程的虚拟地址空间中，代码段和数据段是私有的文件映射，未初始化数据段、堆和栈是私有的匿名映射。

《内存映射的原理详解》

应用编程接口-->内存管理子系统提供如下常用的系统调用：

a. mmap()来创建内存映射。

```
void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset);
```

b. mremap()来扩大或缩小已经存在的内存映射，可能同时移动。

```
void * mremap(void *old_address, size_t old_size , size_t new_size, int flags);
```

c. munmap()用来删除内存映射。

```
int munmap(void *addr, size_t length);
```

d. brk()用来设备堆的上界。

```
int brk(void *addr);
```

e. remap_file_pages()用来创建非线性的文件映射，即文件区间和虚拟地址空间之间的映射不是线性关系，现在已被废除。

```
int remap_file_pages(void *addr, size_t size, int prot,size_t pgoff, int flags);
```

f.mprotect()用来设置虚拟内存区域的访问权限。

int mprotect(void *addr, size_t len, int prot);

g.madvise()用来向内核提供提出内存使用的建议，应用程序告诉内核期望怎样使用指定的虚拟内存区域，以便内核可以选择适合的预读和缓存技术。

int madvise(void *addr, size_t length, int advice);

4、内存映射数据结构

1）虚拟内存区域：是分配给进程的一个虚拟地址范围，内核使用结构体 vm_area_struct 描述虚拟内存区域：

include > linux > C mm_types.h > ...

```
/*
 * This struct defines a memory VMM memory area. There is one of these
 * per VM-area/task.  A VM area is any part of the process virtual memory
 * space that has a special rule for the page-fault handlers (ie a shared
 * library, the executable area etc).
 */
struct vm_area_struct {
    /* The first cache line has the info for VMA tree walking. */

    unsigned long vm_start;     /* Our start address within vm_mm. */
    unsigned long vm_end;       /* The first byte after our end address
                                   within vm_mm. */

    /* linked list of VM areas per task, sorted by address */
    struct vm_area_struct *vm_next, *vm_prev;

    struct rb_node vm_rb;

```

5、创建内存映射

C 标准库封装了函数 mmap 用来创建内存映射，内核提供 POSIX 标准定义的系统调用 mmap 如下：

6、虚拟内存提交策略

7、删除内存映射

二、内存模型/引导内存分配器

1、体系结构（NUMA、SMP）

2、内存模型（平坦内存、不连续内存、稀疏内存）

3、三级结构（node->zone->page）

4、bootmem 分配器

5、memblock 分配器

6、物理内存数据信息

三、伙伴分配器/块分配器

1、伙伴分配器基本知识

2、分区的伙伴分配器（数据结构，首选/备用区域类型、区域水线）

3、每处理器页集合

4、分配页/释放页

5、块分配器编程接口实战

6、slab 分配器、slub 分配器、slob 分配器

一、kmalloc/vmalloc 系统调用

1、kmalloc()用于申请较小的/连续的物理内存，以字节为单位进行分配（在头文件</include/linux/slab.h>）麒麟操作系统（UbuntuKylin）20.00 以上

函数原型如下：

void *kmalloc(size_t size, int flags) // 分配的内存物理地址上连续，虚拟地址上自然连续

include > linux > C slab.h > ...

```c


static __always_inline void *kmalloc(size_t size, gfp_t flags)
{
    if (__builtin_constant_p(size)) {
        if (size > KMALLOC_MAX_CACHE_SIZE)
            return kmalloc_large(size, flags);
#ifndef CONFIG_SLOB
        if (!(flags & GFP_DMA)) {
            int index = kmalloc_index(size);

            if (!index)
                return ZERO_SIZE_PTR;

            return kmem_cache_alloc_trace(kmalloc_caches[index],
                    flags, size);
        }
#endif
    }
    return __kmalloc(size, flags);
}
```

2、void kfree(const void *ptr)释放由 kmalloc()分配出来的内存块。

include > linux > C slab.h > ...

```c
/*
 * Common kmalloc functions provided by all allocators
 */
void * __must_check __krealloc(const void *, size_t, gfp_t);
void * __must_check krealloc(const void *, size_t, gfp_t);
void kfree(const void *);
void kzfree(const void *);
size_t ksize(const void *);

#ifdef CONFIG_HAVE_HARDENED_USERCOPY_ALLOCATOR
const char *__check_heap_object(const void *ptr, unsigned long n,
                struct page *page);
#else
static inline const char *__check_heap_object(const void *ptr,
                unsigned long n,
                struct page *page)
{
    return NULL;
}
#endif
```

```c
mm > C slab.c > ...

void kfree(const void *objp)
{
    struct kmem_cache *c;
    unsigned long flags;

    trace_kfree(_RET_IP_, objp);

    if (unlikely(ZERO_OR_NULL_PTR(objp)))
        return;
    local_irq_save(flags);
    kfree_debugcheck(objp);
    c = virt_to_cache(objp);
    debug_check_no_locks_freed(objp, c->object_size);

    debug_check_no_obj_freed(objp, c->object_size);
    __cache_free(c, (void *)objp, _RET_IP_);
    local_irq_restore(flags);
}
EXPORT_SYMBOL(kfree);
```

3、vmalloc()用于申请较大的内存空间，虚拟内存是连续的，以字节为单位进行分配（在头文件</include/linux/vmalloc.h>）

函数原型如下：

void *vmalloc(unsigned long size) // 分配的内存虚拟地址上连续，物理地址不连续

include > linux > C vmalloc.h > ...

```c
extern void *vmalloc(unsigned long size);
extern void *vzalloc(unsigned long size);
extern void *vmalloc_user(unsigned long size);
extern void *vmalloc_node(unsigned long size, int node);
extern void *vzalloc_node(unsigned long size, int node);
extern void *vmalloc_exec(unsigned long size);
extern void *vmalloc_32(unsigned long size);
extern void *vmalloc_32_user(unsigned long size);
extern void *__vmalloc(unsigned long size, gfp_t gfp_mask, pgprot_t prot);
extern void *__vmalloc_node_range(unsigned long size, unsigned long align,
            unsigned long start, unsigned long end, gfp_t gfp_mask,
            pgprot_t prot, unsigned long vm_flags, int node,
            const void *caller);
#ifndef CONFIG_MMU

extern void *__vmalloc_node_flags(unsigned long size, int node, gfp_t flags);

static inline void *__vmalloc_node_flags_caller(unsigned long size, int node,
                    gfp_t flags, void *caller)
{
    return __vmalloc_node_flags(size, node, flags);
}
#else
extern void *__vmalloc_node_flags_caller(unsigned long size,
                int node, gfp_t flags, void *caller);
#endif
```

mm > C vmalloc.c > ...

```c

void *vmalloc(unsigned long size)
{
    return __vmalloc_node_flags(size, NUMA_NO_NODE,
                GFP_KERNEL);
}
EXPORT_SYMBOL(vmalloc);
```

mm > C vmalloc.c > ...

```c
static inline void *__vmalloc_node_flags(unsigned long size,
                int node, gfp_t flags)
{
    return __vmalloc_node(size, 1, flags, PAGE_KERNEL,
                node, __builtin_return_address(0));
}
```

```c
mm > C vmalloc.c > ...

static void *__vmalloc_node(unsigned long size, unsigned long align,
                    gfp_t gfp_mask, pgprot_t prot,
                    int node, const void *caller)
{
    return __vmalloc_node_range(size, align, VMALLOC_START, VMALLOC_END,
                    gfp_mask, prot, 0, node, caller);
}
```

```c
mm > C vmalloc.c > ...

void *__vmalloc_node_range(unsigned long size, unsigned long align,
            unsigned long start, unsigned long end, gfp_t gfp_mask,
            pgprot_t prot, unsigned long vm_flags, int node,
            const void *caller)
{
    struct vm_struct *area;
    void *addr;
    unsigned long real_size = size;

    size = PAGE_ALIGN(size);
    if (!size || (size >> PAGE_SHIFT) > totalram_pages)
        goto fail;

    area = __get_vm_area_node(size, align, VM_ALLOC | VM_UNINITIALIZED |
                vm_flags, start, end, node, gfp_mask, caller);
    if (!area)
        goto fail;

    addr = __vmalloc_area_node(area, gfp_mask, prot, node);
    if (!addr)
        return NULL;
```

一般情况下，只有硬件设备才需要物理地址连续的内存，因为硬件设备往往存在于 MMU 之外，根本不了解虚拟地址；但为了性能上的考虑，内核中一般使用 kmalloc()，而只有在需要获得大块内存时才使用 vmalloc()，例如当模块被动态加载到内核当中时，就把模块装载到由 vmalloc()分配 的内存上。

4、void vfree(void *addr) // 这个函数可以睡眠，因此不

能从中断上下文调用。

```
mm > C vmalloc.c > ...
void vfree(const void *addr)
{
    BUG_ON(in_nmi());

    kmemleak_free(addr);

    if (!addr)
        return;
    if (unlikely(in_interrupt()))
        __vfree_deferred(addr);
    else
        __vunmap(addr, 1);
}
EXPORT_SYMBOL(vfree);
```

5、malloc(), vmalloc()和 kmalloc()区别

kmalloc 和 vmalloc 是分配的是内核的内存，malloc 分配的是用户的内存；

kmalloc 保证分配的内存在物理上是连续的，vmalloc 保证的是在虚拟地址空间上的连续，malloc 不保证任何东西；

kmalloc 能分配的大小有限，vmalloc 和 malloc 能分配的大小相对较大；

内存只有在要被 DMA 访问的时候才需要物理上连续；

vmalloc 比 kmalloc 要慢；

kmalloc 对应于 kfree，可以分配连续的物理内存；

vmalloc 对应于 vfree，分配连续的虚拟内存，但是物理上不一定连续。

vmalloc 分配内存的时候逻辑地址是连续的，但物理地址一般是不连续的，适用于那种一下需要分配大量内存的情况，如 insert 模块的时候。这种分配方式性能不入 kmalloc。

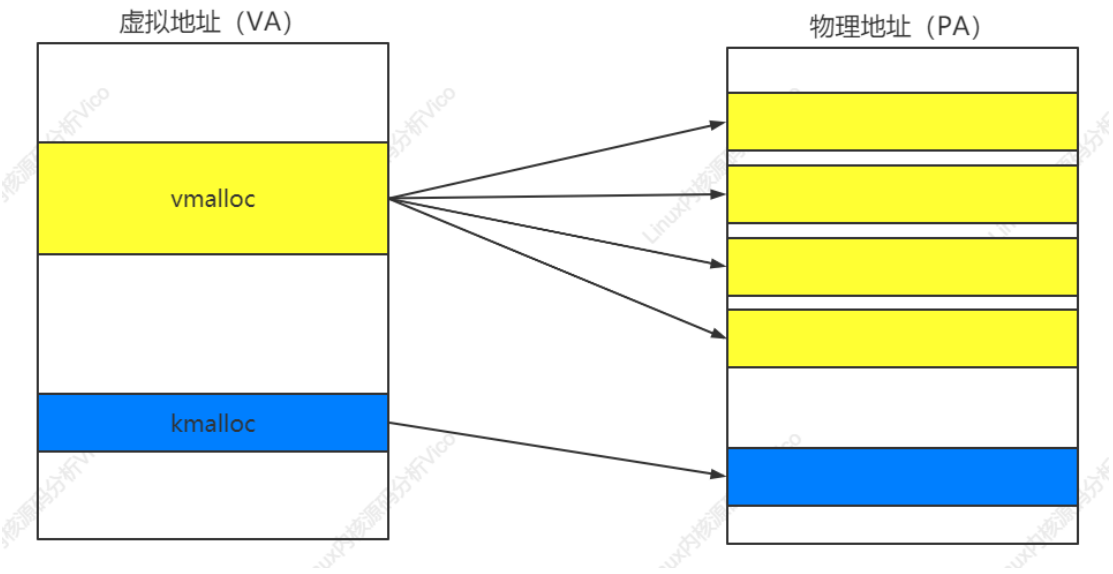
kmalloc 分配内存是基于 slab，因此 slab 的一些特性包括着色，对齐等都具备，性能较好。物理地址和逻辑地址都是连续的。

6、最主要的区别是

在设备驱动程序或者内核模块中动态开辟内存，不是用 malloc，而是 kmalloc ,vmalloc，释放内存用的是 kfree,vfree，kmalloc 函数返回的是虚拟地址(线性地址). kmalloc 特殊之处在于它分配的内存是物理上连续的,这对于要进行 DMA 的设备十分重要. 而用 vmalloc 分配的内存只是线性地址连续,物理地址不一定连续,不能直接用于 DMA。vmalloc 函数的工作方式类似于 kmalloc，只不过前者分配的内存虚拟地址是连续的，而物理地址则无需连 续。通过 vmalloc 获得的页必须一个一个地进行映射，效率不高， 因此，只在不得已(一般

是为了获得大块内存)时使用。vmalloc 函数返回一个指针，指向逻辑上连续的一块内存区，其大小至少为 size。在发生错误 时，函数返回 NULL。vmalloc 可能睡眠，因此，不能从中断上下文中进行调用，也不能从其它不允许阻塞的情况下调用。要释放通过 vmalloc 所获 得的内存，应使用 vfree 函数

vmalloc 和 kmalloc 的分配内存结构图如下：



1：vmalloc 分配的一般为高端内存，只有当内存不够的时候才分配低端内存；kmallco 从低端内存分配；

2：vmalloc 分配的物理地址一般不连续，而 kmalloc 分配的地址连续，两者分配的虚拟地址都是连续的；

3：vmalloc 分配的一般为大块内存，而 kmalloc 一

般分配的为小块内存，（一般不超过 128k)。

二、处理器缓存/内存屏障

1、缓存结构

2、缓存策略

3、缓存维护

4、SMP 缓存一致性

5、编译器屏障

6、处理器内存屏障

7、MMIO 写屏障

8、隐含内存屏障

三、页表缓存/页回收机制

1、页表缓存概念

2、TLB 表项格式及管理

3、地址空间标识符

4、虚拟机标识符

5、页回收数据结构

6、发起页回收

7、计算扫描的页数

8、收缩活动页链表

9、页交换

10、回收 slab 缓存

四、Linux 缺页中断处理/反碎片技术

【Linux 缺页中断处理】

A、什么是缺页中断？

进程线性地址空间里的页面不必常驻内存，在执行一条指令时，如果发现他要访问的页没有在内存中（即存在位为 0），那么停止该指令的执行，并产生一个页不存在的异常，对应的故障处理程序可通过从外存加载该页的方法来排除故障，之后，原先引起的异常的指令就可以继续执行，而不再产生异常。

B、页面调度算法

将新页面调入内存时，如果内存中所有的物理页

都已经分配出去，就按照某种策略来废弃整个页面，将其所占据的物理页释放出来；

1、请求调页中断：

进程线性地址空间里的页面不必常驻内存，例如进程的分配请求被理解满足，空间仅仅保留 vm_area_struct 的空间，页面可能被交换到后援存储器，或者写一个只读页面(COW)。Linux 采用请求调页技术来解决硬件的缺页中断异常，并且通过预约式换页策略。

主缺页中断和次缺页中断，费时的需要从磁盘读取数据时就会产生主缺页中断。

每种 CPU 结构提供一个 do_page_fault (struct pt_regs *regs, error_code)处理缺页中断，该函数提供了大量信息，如发生异常地址，是页面没找到还是页面保护错误，是读异常还是写异常，来自用户空间还是内核空间。它负责确定异常类型及异常如何被体系结构无关的代码处理。下图是 Linux 缺页中断处理流程：

图-Linux 缺页中断处理

一旦异常处理程序确定异常是有效内存区域中的有效缺页中断，将调用体系结构无关的函数

handle_mm_fault()。如果请求页表项不存在，就分配请求的页表项，并调用 handle_pte_fault()。

第一步调用 pte_present 检查 PTE 标志位，确定其是否在内存中，然后调用 pte_none()检查 PTE 是否分配。如果 PTE 还没有分配的话，将调用 do_no_page()处理请求页面的分配，否则说明该页已经交换到磁盘，也是调用 do_swap_page()处理请求换页。如果换出的页面属于虚拟文件则由 do_no_page()处理。

第二步确定是否写页面。如果 PTE 写保护，就调用 do_swap_page()，因为这个页面是写时复制页面。COW 页面识别方法：页面所在 VMA 标志位可写，但相应的 PTE 确不是可写的。如果不是 COW 页面，通常将之标志为脏，因为它已经被写过了。

第三步确定页面是否已经读取及是否在内存中，但还会发生异常。这是因为在某些体系结构中没有 3 级页表，在这种情况下建立 PTE 并标志为新即可。

2、请求页面分配：

第一次访问页面，首先分配页面，一般由 do_no_page()填充数据。如果父 VMA 的 vm_area_struct

一>vm_ops 提供了 nopage()函数，则用它填充数据；否则调用 do_anonymous_page()匿名函数来填充数据。如果被文件或设备映射，如果时文件映射，filemap_nopage()将替代 nopage()函数，如果由虚拟文件映射而来，则 shmem_nopage()。每种设备驱动将提供不同的 nopage()函数，该函数返回 struct page 结构。

3、请求换页：

将页面交换至后援存储器后，函数 do_swap_page()负责将页面读入内存，将在后面讲述。通过 PTE 的信息就足够查找到交换的页面。页面交换出去时，一般先放到交换高速缓存中。

缺页中断时如果页面在高速缓存中，则只要简单增加页面计数，然后把它放到进程页表中并计数次缺页中断发生的次数。如果页面仅存在磁盘中，Linux 将调用 swapin_readahead()读取它及后续的若干页面。

4、页面帧回收：

除了 slab 分配器，系统中所有正在使用的页面都存放在页面高速缓存中，并由 page->lru 链接在一起。

Slab 页面不存放到高速缓存中因为基于被 slab 使用的对象对页面计数很困难。除了查找每个进程页表外没有其他方法能把 struct page 映射为 PTE，查找页表代价很大。如果页面高速缓存中存在大量的进程映射页面，系统将会遍历进程页表，通过 swap_out()函数交换出页面直到有足够的页面空闲，而共享页会给 swap_out()带来问题。如果一个页面是共享的，同时一个交换项已经被分配，PTE 就会填写所需信息以便在交换分区里重新找到该页并将引用计数减 1。只有引用计数为 0 时该页才能被替换出去。

内存和磁盘缓存申请越来越多的页面但确无法判断如何释放进程页面，请求分页机制在进程页面缺失时申请新页面，但它却不能强制释放进程不再使用的页面。The Page Frame Reclaiming Algorithm(PFRA)页面回收算法用于从用户进程和内核 cache 中回收页面放到伙伴系统的空闲块列表中。PFRA 必须在系统空闲内存达到某个最低限度时进行页面回收，回收的对象必须是非空闲页面。

可将系统页面划分为四种：

1)Unreclaimable 不可回收的，包括空闲页面、保留页面设置了 PG_reserved 标志、内核动态分配的页面、

进程内核栈的页面、设置了 PG_locked 标志的临时锁住的页面、设置了 VM_LOCKED 标志的内存页面。

2)Swappable 可交换的页面，用户进程空间的匿名页面(用户堆栈)、tmpfs 文件系统的映射页面（入 IPC 共享内存页面），页面存放到交换空间。

3)Syncable 可同步的页面，入用户态地址空间的映射页面、保护磁盘数据的页面缓存的页面、块设备缓冲页、磁盘缓存的页面（入 inode cache），如果有必要的话，需同步磁盘映像上的数据。

4)Discardable 可丢弃的页面，入内存缓存中的无用页面（slab 分配器中的页面）、dentry cache 的页面。

PFRA 算法是基于经验而非理论的算法，它的设计原则如下：

1)首先释放无损坏的页面。进程不再引用的磁盘和内存缓存应该先于用户态地址空间的页面释放。

2)标志所有进程态进程的页面为可回收的。

3)多进程共享页面的回收，要先清除引用该页面的进程页表项，然后再回收。

4)回收“不在使用的”页面。PFRA 用 LRU 链表把进

程划分为 in-use 和 unused 两种，PFRA 仅回收 unused 状态的页面。Linux 使用 PTE 中的 Accessed 比特位实现非严格的 LRU 算法。

页面回收通常在三种情况下执行：

1)系统可用内存比较低时进行回收(通常发生在申请内存失败)。

2)内核进入 suspend-to-disk 状态时进行回收。

3)周期性回收，内核线程周期性激活并在必要时进行页面回收。

Low on memory 回收有以下几种情形：

1)_ _getblk( )调用的 grow_buffers( )函数分配新缓存页失败；

2)create_empty_buffers( ) 调用的 alloc_page_buffers( )函数为页面分配临时的 buffer head 失败；

3)_ _alloc_pages( )函数在给定内存区里分配一组连续的页面帧失败。

周期性回收涉及的两种内核线程：

1)Kswapd 内核线程在内存区中检测空闲页面是否低于 pages_high 的门槛值；

2)预定义工作队列中的事件内核线程，PFRA 周期性调度该工作队列中的 task 回收 slab 分配器中所有空闲的 slab；

所有用户空间进程和页面缓存的页面被分为活动链表和非活动链表，统称 LRU 链表。每个区描述符中包括 active_list 和 inactive_list 两个链表分别将这些页面链接起来。nr_active 和 nr_inactive 分别表示这两种页面数量，lru_lock 用于同步。页描述符中的 PG_lru 用于标志一个页面是否属于 LRU 链表，PG_active 用于标志页面是否属于活动链表，lru 字段用于把 LRU 中的链表串起来。活动链表和非活动链表的页面根据最近的访问情况进行动态调整。PG_referenced 标志就是此用途。

处理 LRU 链表的函数有：

add_page_to_active_list() 、

add_page_to_inactive_list() 、 activate_page() 、 lru_cache_add()、lru_cache_add_active()等，这些函数比较简单。

shrink_active_list ( )用于将页表从活动链表移到非活动链表。该函数在 shrink_zone()函数执行用户地址空间的页面回收时执行。

5、交换分区：

系统可以有 MAX_SWAPFILES 的交换分区，每个分区可放在磁盘分区上或者普通文件里。每个交换区由一系列页槽组成。每个交换区有个 swap_header 结构描述交换区版本等信息。每个交换区有若干个 swap_extent 组成，每个 swap_extent 是连续的物理区域。对于磁盘交换区只有一个 swap_extent，对于文件交换区则由多个 swap_extent 组成，因为文件并不是放在连续的磁盘块上的。mkswap 命令可以创建交换分区。

图 交换分区结构

图 交换页结构

swp_type() 和 swp_offset()函数根据页槽索引和交换区号得到 type 和 offset 值，函数 swp_entry(type,offset)得到交换槽。最后一位总是清 0 表示页不在 RAM 上。槽最大 224(64G)。第一个可用槽索引为 1。槽索引不能全为 0。

一个页面可能被多个进程共用，它可能被从一个进程地址空间换出但仍然在物理内存上，因此一个页面可能被多次换出。但物理上仅第一次被换出并存储在交换区上，接下来的换出操作只是增加 swap_map 的引用计数。swap_duplicate(swp_entry_t entry)的功能正是用户尝试换出一个已经换出的页面。

6、交换缓存：

多个进程同时换进一个共享匿名页时或者一个进程换入一个正被 PFRA 换出的页时存在竞争条件，引入交换缓存解决这种同步问题。通过 PG_locked 标志可以保证对一个页的并发的交换操作只作用在一个页面上，从而避免竞争条件。

7、页面回收算法描述：

下图是各种情况下进行页面回收时的函数调用关系图。可以看出最终调用函数为 cache_reap()、shrink_slab()和 shrink_list()。cache_reap()用于周期性回收 slab 分配器中的无用 slab。shrink_slab()用于回收磁盘缓存的页面。shrink_list()是页面回收的核心函数，在最新代码中该函数名改为 shrink_page_list()。下面会重点讲解。

图中 shrink_caches() 最新函数名为 shrink_zones() 、 shrink_cache() 最新函数名为 shrink_inactive_list()。其他函数不变。

图 PFRA 函数结构调用关系

低内存回收页面：

如上图所示，当内存分配失败时，内核调用 free_more_memory(),该函数首先调用 wakeup_bdflush( ) 唤醒 pdflush 内核线程触发写操作，从磁盘页面缓冲中写 1024 个 dirty 页面到磁盘上以释放包含缓冲、缓冲头和 VFS 的数据结构所占用的页表；然后进行系统调用 sched_yield( )，以使 pdflush 线程能够有机会运行；最后

该函数循环遍历系统节点，对每个节点上的低内存区(ZONE_DMA 和 ZONE_NORMAL)调用 try_to_free_pages( )函数。

try_to_free_pages(struct zone **zones, gfp_t gfp_mask)函数的目标是通过循环调用 shrink_slab()和 shrink_zones()释放至少 32 个页帧，每次调用增加优先级参数，初始优先级是 12，最高为 0。如果循环 13 次，仍然没有释放掉 32 个页面，则 PFRA 进行内存异出保护：调用 out_of_memory()函数选择一个进程回收它所有的页面。

shrink_zones(int priority, struct zone **zones, struct scan_control *sc)函数对 zones 列表中每个区调用 shrink_zone()函数。调用 shrink_zone()前，先用 sc->priority 的值更新 zone 描述符中的 prev_priority，如果 zone->all_unreclaimable 字段不为 0 且优先级不是 12，则不对该区进行页面回收。

shrink_zone(int priority, struct zone *zone, struct scan_control *sc)函数尝试回收 32 个页面。该函数循环进行 shrink_active_list()和 shrink_inactive_list 的操作以达到目标。该函数流程如下：

1) atomic_inc(&zone->reclaim_in_progress)增加区的回

收计数；

2) 增加 zone->nr_scan_active，根据优先级，增加范围是 zone->nr_active/212 to zone->nr_active/20 。如果 zone->nr_scan_active >= 32 则赋给 nr_active 变量，同时 zone->nr_scan_active 设为 0，否则 nr_active＝0；

3) zone->nr_scan_inactive 和 nr_inactive 做同样处理；

4) 如果 nr_active 和 nr_inactive 不同时为空，则进行 while 循环进行 5、6 步操作：

5) 如果 nr_active 非 0，则从 active 链表移动某些页面到 inactive 链表：

```
nr_to_scan = min(nr_active,(unsigned long)sc->swap_cluster_max);
nr_active -= nr_to_scan;
shrink_active_list(nr_to_scan, zone, sc, priority);
```

6) 如果 nr_inactive 非 0，则回收 inactive 链表中的页面：

```
nr_to_scan = min(nr_inactive,(unsigned long)sc->swap_cluster_max);
nr_inactive -= nr_to_scan;
```

nr_reclaimed += shrink_inactive_list(nr_to_scan, zone, sc);

7) atomic_dec(&zone->reclaim_in_progress)减小回收计数，并返回回收页面数 nr_reclaimed；

shrink_inactive_list(unsigned long max_scan, struct zone *zone, struct scan_control *sc)函数从区的 inactive 链表中抽取一组页面，放到临时链表中，调用 shrink_page_list()对链表中的每个页面进行回收。下面是 shrink_inactive_list()主要步骤：

1) 调用 lru_add_drain()将当前 CPU 上 pagevec 结构的 lru_add_pvecs 和 lru_add_active_pvecs 中的页面分别移到活动链表和非活动链表中；

2) 获取 LRU 锁 spin_lock_irq(&zone->lru_lock);

3) 最多扫描 max_scan 个页面，对每个页面增加使用计数，检查该页面是否正被释放到伙伴系统中，将该页面移动一个临时链表中；

4) 从 zone->nr_inactive 中减去移到临时链表中的页面数；

5) 增加 zone->pages_scanned 计数；

6) 释放 LRU 锁：spin_unlock_irq(&zone->lru_lock)；

7) 对临时链表调用 shrink_page_list(&page_list, sc)回收页面；

8) 增加 nr_reclaimed 计数；

9) 获取 LRU 锁 spin_lock(&zone->lru_lock)；

10) 将 shrink_page_list(&page_list, sc)没有回收掉的页面重新添加到 active 链表和 inactive 链表中。该函数在回收过程中可能会设置 PG_active 标志，所以也要考虑往 active 链表中添加。

11) 如果扫描页面数 nr_scanned 小于 max_scan 则循环进行 3～10 的操作；

12) 返回回收的页面数；

shrink_page_list(struct list_head *page_list, struct scan_control *sc)做真正的页面回收工作，该函数流程如下：

图 shrink_page_list()页面回收逻辑处理流程

1) 调用 cond_resched()进行条件调度；

2) 循环遍历 page_list 中每个页面，从列表中移出该页面描述符并回收该页面，如果回收失败，则把该页面插入一个局部链表中；该步流程参见流程图。

- 调用 cond_resched() 进行条件调度；
- 从 LRU 链表中取出第一个页面并从 LRU 链表中删除；
- 如果页面被锁定，这调过该页面，该页加到临时链表中；
- 如果页面不能部分回收并且页面是进程页表的映射，这跳过该页；
- 如果进程是回写的 dirty 页面，则跳过；
- 如果页面被引用并且页面映射在使用，这跳过并激活该页面，以便后面放入 active 列表；
- 如果是匿名页面且不在交换区中，这调用 add_to_swap()为该页面分配交换区空间并把该页加到交换缓存中；
- 如果页面是进程空间映射并且页面映射地址非空，则调用 try_to_unmap()移除该页面的页表映射；

l 如果页面为 dirty 页面并且无引用、交换可写、且是 fs 文件系统映射，调用 pageout()写出该页面。

3) 循环结束，把局部链表中的页面移回到 page_list 链表中；

4) 返回回收页面数。

每个页面帧处理后只有三种结果：

1) 通过调用 free_code_page()页面被释放到伙伴系统中，页面被有效回收；

2) 页面没有回收，被重新插入到 page_list 链表中，并且认为该页面在将来可能会被再次回收，因而清除 PG _active 标志，以便在后面加入到 inactive 链表中；

3) 页面没有回收，被重新插入到 page_list 链表中，并且认为该页面在可预见的将来不会被再次回收，因而设置 PG _active 标志，以便在后面加入到 active 链表中

回收一个匿名页面时，该页面必须添加到交换缓存中，交换区中必须为其预留一个新页槽。如果页面在某些进程的用户态地址空间中，shrink_page_list()调用 try_to_unmap 定位所有执向该页面帧的进程 PTE 项，只

有返回成功时才继续回收；如果页面是 dirty 状态，必须要写到磁盘上才能回收，这需要调用 pageout()函数，回收只有在 pageout()很快完成写操作或者不必进行写操作时才继续进行；如果页面保护 VFS buffers，则调用 try_to_release_page()释放 buffer heads。

最后如果上面都进展顺利的话， shrink_page_list()函数检查页的引用计数：如果值正好为 2，则一个为页面缓存或交换缓存，另一个是 PFRA 自身(shrink_inactive_page()函数中增加该值)。这种情况下，该页面可以回收，并且它不为 dirty。根据页面 PG_swapcache 标志，页面从页面缓存或交换缓存中移除；然后调用 free_code_page()。

换出页面

add_to_swap(struct page * page, gfp_t gfp_mask)换出操作首先是为页面分配交换页槽，并分配交换缓存；步骤如下：

1) get_swap_page()为换出页面预留交换槽位；

2) 调用__add_to_swap_cache()传入槽索引、页描述符和 gfp 标志将页面加到交换缓存中，并标记为 dirty；

3) 设置页面 PG _uptodate 和 PG_dirty 标志，以便 shrink_inactive_page()能够强制将页面写到磁盘上；

4) 返回；

try_to_unmap(struct page *page, int migration)，换出操作第二步，在 add_to_swap 后面调用，该函数查找所有用户页表中指向该匿名页帧的页表项，并在 PTE 中设置换出标志。

Page_out()换出操作第三步将 dirty 页面写到磁盘：

1) 检查页面缓存或交换缓存中的页，并查看该页面是否近被页面缓存或交换缓存占有；如果失败，返回 PAGE_KEEP。

2) 检查 address_space 对象的 writepage 方法是否定义，如没有返还 PAGE_ACTIVATE；

3) 检查当前进程是否可以发送写请求到当前映射地址空间对象对应的块设备上请求队列上。

4) SetPageReclaim(page)设置页面回收标志；

5) 调用 mapping->a_ops->writepage(page, &wbc)进行

写操作，如果失败则清除回收标志；

6) 如果 PageWriteback(page)失败，页面没有写回，则清除回收标志 ClearPageReclaim(page)；

7) 返回成功；

对于交换分区，writepage 的实现函数是 swap_writepage()，该函数流程如下:

1) 检查是否有其他进程引用该页面，如果没有，从交换缓存中移除该页面返回 0；

2) get_swap_bio()分配初始化 bio 描述符，该函数从交换页标志中找到交换区，然后遍历交换扩展链表找到页槽的起始磁盘分区。bio 描述符包含对一个页面的请求，并设置完成方法为 end_swap_bio_write()。

3) set_page_writeback(page)设置页面 writeback 标志，unlock_page()该页面解锁；

4) submit_bio(rw, bio)向块设备提交 bio 描述符进行写操作；

5) 返回；

一旦写操作完成，end_swap_bio_write()被执行。该函数唤醒等待页面 PG_writeback 标志清除的进程，清除 PG_writeback 标志，是否 bio 描述符。

换入页面

换入页面操作发生在一个进程访问被换出到磁盘上的页面时。当下列条件发生时页面出错处理程序会进行换入操作：

1) 包含引发异常的地址的页面是一个当前进程内存区域的有效页面；

2) 该页面不在内存中，PTE 的页面 present 表示被清 0；

3) 与页面相关的 pte 不为 null，Dirty 位被清 0，这意味着该 pte 包含换出页的标志；

当上述条件同时满足时，hand_pte_fault() 调用 do_swap_page()函数换入请求页面。

do_swap_page(struct mm_struct *mm, struct vm_area_struct *vma,

unsigned long address, pte_t *page_table, pmd_t *pmd,

int write_access, pte_t orig_pte)

该函数处理流程如下：

1) entry = pte_to_swp_entry(orig_pte)得到交换槽位信息；

2) page = lookup_swap_cache(entry)查看交换槽对应的页面是否存在于交换缓存中，如果是则跳到第 6 步；

3) 调用 swapin_readahead(entry, address, vma)从交换区中读取一组页面，对每个页面调用 read_swap_cache_async()读取该页面；

4) 对于进程访问异常的页面再次调用 read_swap_cache_async()。因为 swapin_readahead 调用可能失败，在它成功的情况下 read_swap_cache_async()发现该页面在交换缓存里，很快返回；

5) 如果页面还是没有在交换缓存中，可能存在其他内核控制路径已经把该页面换入。比较 page_table 对应的 pte 内容与 orig_pte 是否相同，如果不同，说明页面已经换入。函数跳出返回。

6) 如果页面在交换缓存中，调用 mark_page_accessed 并锁住该页面；

7) pte_offset_map_lock(mm, pmd, address, &ptl)获取 page_table 对应的 pte 内容，与 orig_pte 比较，判断是否有其他内核控制路径进行换入操作；

8) 测试 PG_uptodate 标志，如果未设置，则出错返回；

9) 增加 mm->anon_rss 的计数；

10) mk_pte(page, vma->vm_page_prot)创建 PTE 并设置标志，插入到进程页表中；

11) page_add_anon_rmap()为该匿名页插入反向映射数据结构的内容；

12) swap_free(entry)释放页槽；

13) 检查交换缓存负载是否达到 50%，如果是，并且该页面仅被触发页面访问异常的进程占有，则从交换缓存中释放该页。

14) 如果 write_access 标志为 1，说明是 COW 写时复制，调用 do_wp_page()拷贝一份该页面；

释放页锁和页面缓存等，并返回结果。

【反碎片技术】

1、虚拟可移动区域

2、内存碎片整理

3、内存碎片防治技术

正文

Linux kernel 组织管理物理内存的方式是 buddy system（伙伴系统），而物理内存碎片正式 buddy system 的弱点之一，为了预防以及解决碎片问题，kernel 采取了一些实用技术，这里将对这些技术进行总结归纳。

回到顶部

1 低内存时整合碎片

从 buddy 申请内存页，如果找不到合适的页，则会进行两步调整内存的工作，compact 和 reclaim。前者是为了整合碎片，以得到更大的连续内存；后者是回收不一定必须占用内存的缓冲内存。这里重点了解 comact，整个流程大致如下：

__alloc_pages_nodemask -> __alloc_pages_slowpath -> __alloc_pages_direct_compact -> try_to_compact_pages -> compact_zone_order -> compact_zone -> isolate_migratepages -> migrate_pages -> release_freepages

并不是所有申请不到内存的场景都会 compact，首先要满足 order 大于 0，并且 gfp_mask 携带__GFP_FS 和__GFP_IO；另外，需要 zone 的剩余内存情况满足一定条件，kernel 称之为“碎片指数”（fragmentation index），这个值在 0~1000 之间，默认碎片指数大于 500 时才能进行 compact，可以通过 proc 文件 extfrag_threshold 来调整这个默认值。fragmentation index 通过 fragmentation_index 函数来计算：

/* * Index is between 0 and 1000 * * 0 => allocation would fail due to lack of memory * 1000 => allocation would fail due to fragmentation */return 1000 - div_u64((1000+(div_u64(info->free_pages * 1000ULL, requested))), info->free_blocks_total)

在整合内存碎片的过程中，碎片页只会在本 zone 的内部移动，将位于 zone 低地址的页尽量移到 zone 的末端。申请新的页面位置通过 compaction_alloc 函数实现。

移动过程又分为同步和异步，内存申请失败后第一次 compact 将会使用异步，后续 reclaim 之后将会使用同步。同步过程只移动当面未被使用的页，异步过程将遍历并等待所有 MOVABLE 的页使用完成后进行移动。

2 按可移动性组织页

按照可移动性将内存页分为以下三个类型：

UNMOVABLE：在内存中位置固定，不能随意移动。kernel 分配的内存基本属于这个类型；

RECLAIMABLE：不能移动，但可以删除回收。例如文件映射内存；

MOVABLE：可以随意移动，用户空间的内存基本属于这个类型。

申请内存时，根据可移动性，首先在指定类型的空闲页中申请内存，每个 zone 的空闲内存组织方式如下：

struct zone {.....struct free_area free_area[MAX_ORDER];.....}struct free_area {struct list_head free_list[MIGRATE_TYPES];unsigned long nr_free;};

当在指定类型的 free_area 申请不到内存时，可以从备用类型挪用，挪用之后的内存就会释放到新指定的类型列表中，kernel 把这个过程称为“盗用”。

备用类型优先级列表如下定义：

static int fallbacks[MIGRATE_TYPES][4] = {[MIGRATE_UNMOVABLE] = { MIGRATE_RECLAIMABLE, MIGRATE_MOVABLE,

MIGRATE_RESERVE },[MIGRATE_RECLAIMABLE] = { MIGRATE_UNMOVABLE, MIGRATE_MOVABLE, MIGRATE_RESERVE },#ifdef CONFIG_CMA[MIGRATE_MOVABLE] = { MIGRATE_CMA, MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE, MIGRATE_RESERVE },[MIGRATE_CMA] = { MIGRATE_RESERVE }, /* Never used */#else[MIGRATE_MOVABLE] = { MIGRATE_RECLAIMABLE, MIGRATE_UNMOVABLE, MIGRATE_RESERVE },#endif[MIGRATE_RESERVE] = { MIGRATE_RESERVE }, /* Never used */#ifdef CONFIG_MEMORY_ISOLATION[MIGRATE_ISOLATE] = { MIGRATE_RESERVE }, /* Never used */#endif};

值得注意的是并不是所有场景都适合按可移动性组织页，当内存大小不足以分配到各种类型时，就不适合启用可移动性。有个全局变量来表示是否启用，在内存初始化时设置：

void __ref build_all_zonelists(pg_data_t *pgdat, struct zone *zone){.....if (vm_total_pages < (pageblock_nr_pages * MIGRATE_TYPES)) page_group_by_mobility_disabled = 1;else page_group_by_mobility_disabled = 0;.....}

如果 page_group_by_mobility_disabled，则所有内存都是不可移动的。其中有个参数决定了每个内存区域至少拥有的页，pageblock_nr_pages，它的定义如下：

#define pageblock_order HUGETLB_PAGE_ORDER#else /* CONFIG_HUGETLB_PAGE *//* If huge pages are not used, group by MAX_ORDER_NR_PAGES */#define pageblock_order (MAX_ORDER-1)#endif /* CONFIG_HUGETLB_PAGE */#define pageblock_nr_pages (1UL << pageblock_order)

在系统初始化期间，所有页都被标记为 MOVABLE：

void __meminit memmap_init_zone(unsigned long size, int nid, unsigned long zone,unsigned long start_pfn, enum memmap_context context){.....if ((z->zone_start_pfn <= pfn)&& (pfn < zone_end_pfn(z))&& !(pfn & (pageblock_nr_pages - 1)))set_pageblock_migratetype(page, MIGRATE_MOVABLE);.....}

其它可移动性类型的页都是后来产生的，也就是前面说的“盗取”。在这种情况发生时，通常会“盗取”fallback 中更高优先级、更大块连续的页，从而避免小碎片的产生。

/* Remove an element from the buddy allocator from the

fallback list */static inline struct page *__rmqueue_fallback(struct zone *zone, int order, int start_migratetype){...../* Find the largest possible block of pages in the other list */for (current_order = MAX_ORDER-1; current_order >= order;--current_order) {for (i = 0;; i++) { migratetype = fallbacks[start_migratetype][i];.....}

可以通过/proc/pageteypeinfo 查看当前系统各种类型的页分布。

3 虚拟可移动内存域

在依据可移动性组织页的技术之前，还有一个方法已经合入 kernel，那就是虚拟内存域：ZONE_MOVABLE。基本思想很简单：把内存分为两部分，可移动的和不可移动的。

enum zone_type {#ifdef CONFIG_ZONE_DMA ZONE_DMA,#endif#ifdef CONFIG_ZONE_DMA32 ZONE_DMA32,#endif ZONE_NORMAL,#ifdef CONFIG_HIGHMEM ZONE_HIGHMEM,#endif ZONE_MOVABLE, __MAX_NR_ZONES};

ZONE_MOVABLE 的启用需要指定 kernel 参数 kernelcore 或者 movablecore，kernelcore 用来指定不可移动的内存

数量，movablecore 指定可移动的内存大小，如果两个都指定，取不可移动内存数量较大的一个。如果都不指定，则不启动。

与其它内存域不同的是 ZONE_MOVABLE 不关联任何物理内存范围，该域的内存取自高端内存域或者普通内存域。find_zone_movable_pfns_for_nodes 用来计算每个 node 中 ZONE_MOVABLE 的内存数量，采用的内存区域通常是每个 node 的最高内存域，在函数 find_usable_zone_for_movable 中体现。

在对每个 node 分配 ZONE_MOVABLE 内存时，kernelcore 会被平均分配到各个 Node：

kernelcore_node = required_kernelcore / usable_nodes;

在 kernel alloc page 时，如果 gfp_flag 同时指定了 __GFP_HIGHMEM 和 __GFP_MOVABLE，则会从 ZONE_MOVABLE 内存域申请内存。